

特 征 选 择



时间：2025年5月

TRANSWARP
星 环 科 技



目录 >

CONTENTS

1 降维和选择的区别

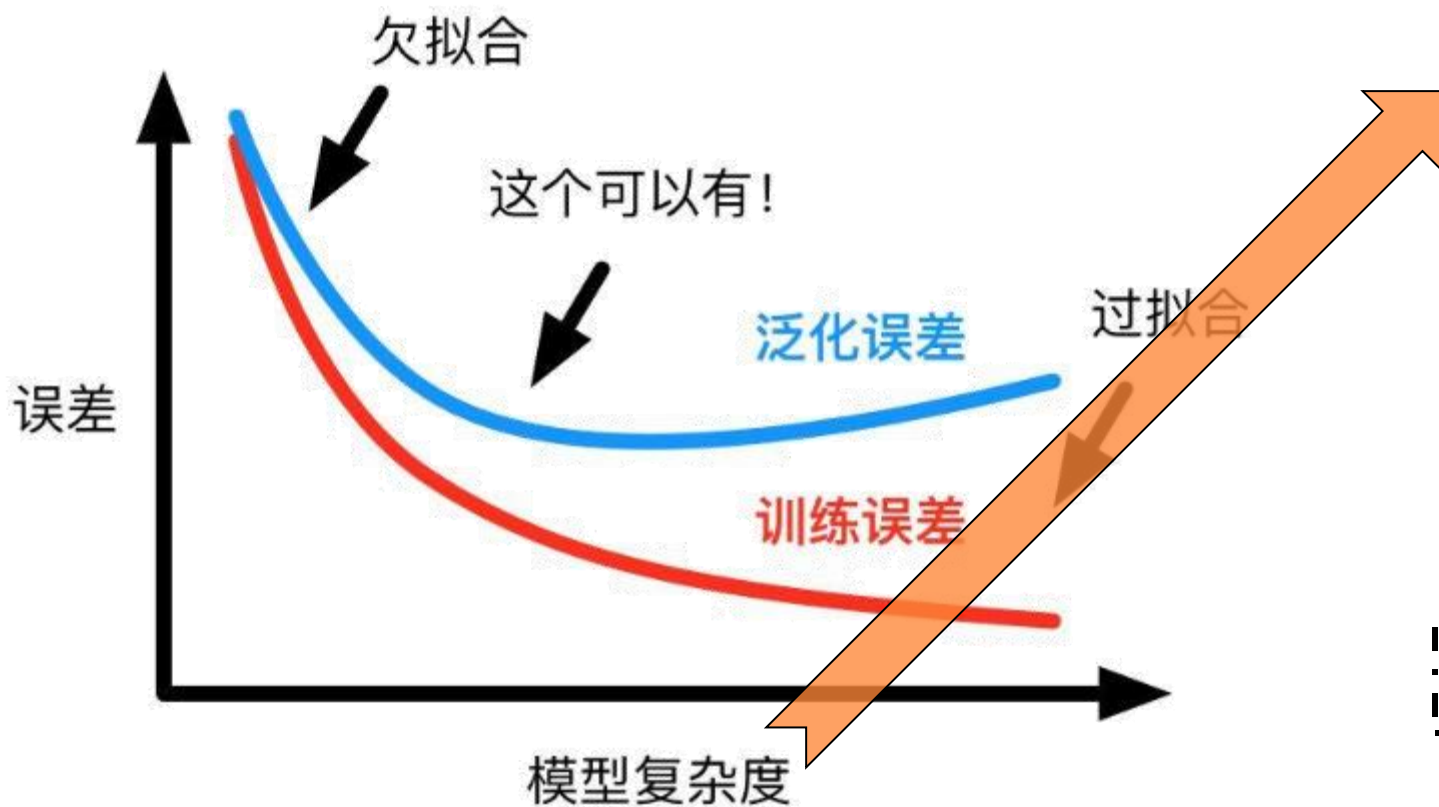
1 特征选择



1 chapter

降维和选择的区别

为什么需要降维？



- 算法方面：
 - 算法参数过多
 - 算法拟合能力太强
 - 算法超参数
 - 算法学习过头

■ 数据方面

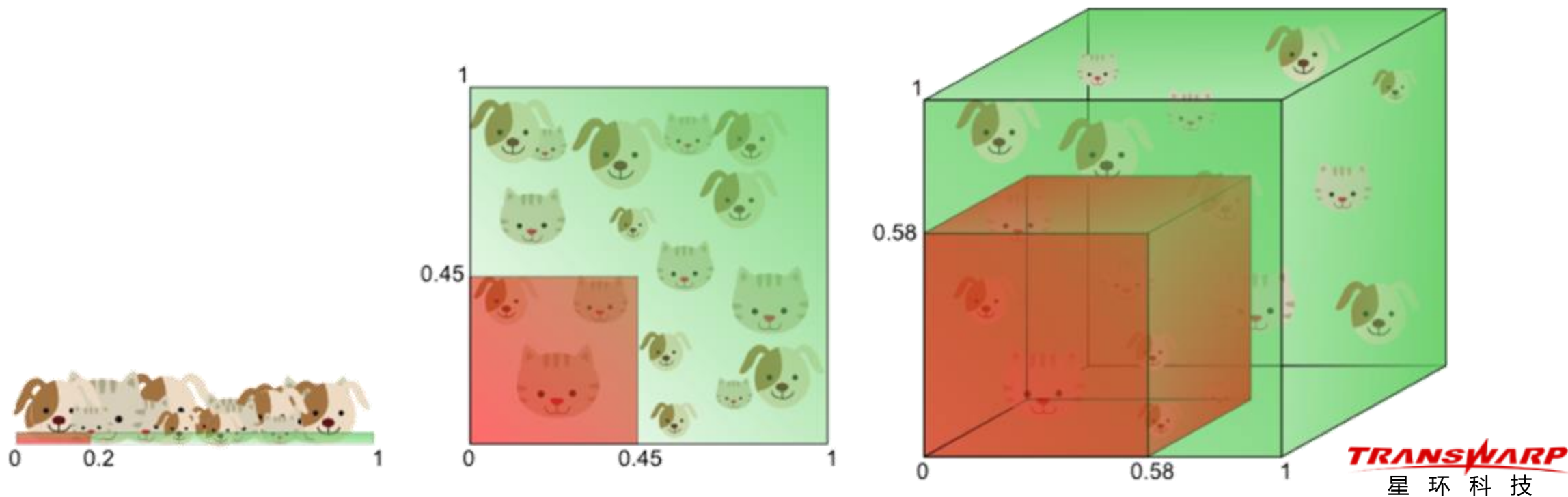
- 数据太少

■ **特征太多**

我们需要**减少特征**

维度灾难

我们假设每个特征的范围是从 0-1，同时每个猫和狗都具有不同的特征。如果我们想使用全数据的 20% 的数据来训练模型，那么在一维的情况下，我们的特征范围只需要取 20% 的范围，也就是 0.2 就够了。当上升到 2 维的情况下，我们需要 45% 的每维特征 ($0.45 \times 0.45 = 0.2$) 才可以覆盖特征空间中的 20%，当上升到 3 维空间时，我们则需要每维 58% 的特征范围 ($0.58^3 = 0.2$)



维度灾难

特征空间

特征1: {A、B、C}

特征2: {红、黄、蓝、紫、橙}

特征3: {1、2、3、4、5、6、7、8}

$$\text{组合数} = 3 \times 5 \times 8 = 120$$

训练样本数为
24

特征1	特征2	特征3	标签y
A	红	1	1
A	红	2	0
A	红	3	1
...	...	...	...
C	橙	4	1
A	橙	3	
B	蓝	1	
...	...	...	

20%的**已知组合样本**训练**模型**

算法模型

使用**模型**预测**未知的组合**

增加特征空间

特征空间

特征1: {A、B、C}

特征2: {红、黄、蓝、紫、橙}

特征3: {1、2、3、4、5、6、7、8}

特征4: {小、中、大}

$$\text{组合数} = 3 \times 5 \times 8 \times 3 = 360$$

训练样
本数为
72

特征空间

特征1: {A、B、C}

特征2: {红、黄、蓝、紫、橙}

特征3: {1、2、3、4、5、6、7、8}

特征4: {小、中、大}

特征45: {A1、A2、...、A100}

$$\text{组合数} = 3 \times 5 \times 8 \times 3 \times 100 = 36000$$

训练样
本数为
7200

特征属性值种类数多

特征多

方差的意义

年龄 (X1)	专业 (X2)	性别 (Y)
19	数学	M
19	IT	M
19	英语	F
19	数学	F
19	数学	M
19	IT	M
19	英语	F
19	数学	F

方差为0，对Y没有任何意义



什么是特征选择?

- 假设有两个特征 x_1 和 x_2 , 我们可以选择**保留 x_2 , 丢掉 x_1**

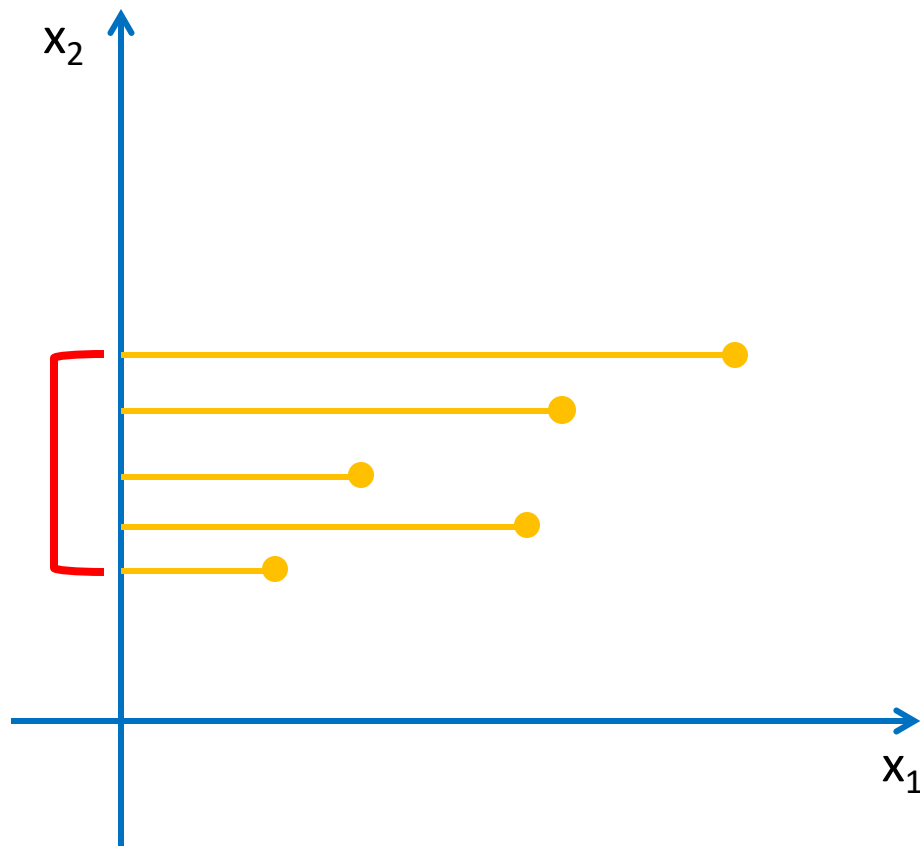
$$(x_1^1, x_2^1)$$

$$(x_1^2, x_2^2)$$

$$(x_1^3, x_2^3)$$

$$(x_1^4, x_2^4)$$

$$(x_1^5, x_2^5)$$



$$(x_1^1, x_2^1)$$

$$(x_1^2, x_2^2)$$

$$(x_1^3, x_2^3)$$

$$(x_1^4, x_2^4)$$

$$(x_1^5, x_2^5)$$

什么是特征选择?

- 我们也可以选择**保留 x_1** ，**丢掉 x_2**

保留的特征是原来的特征中的特征

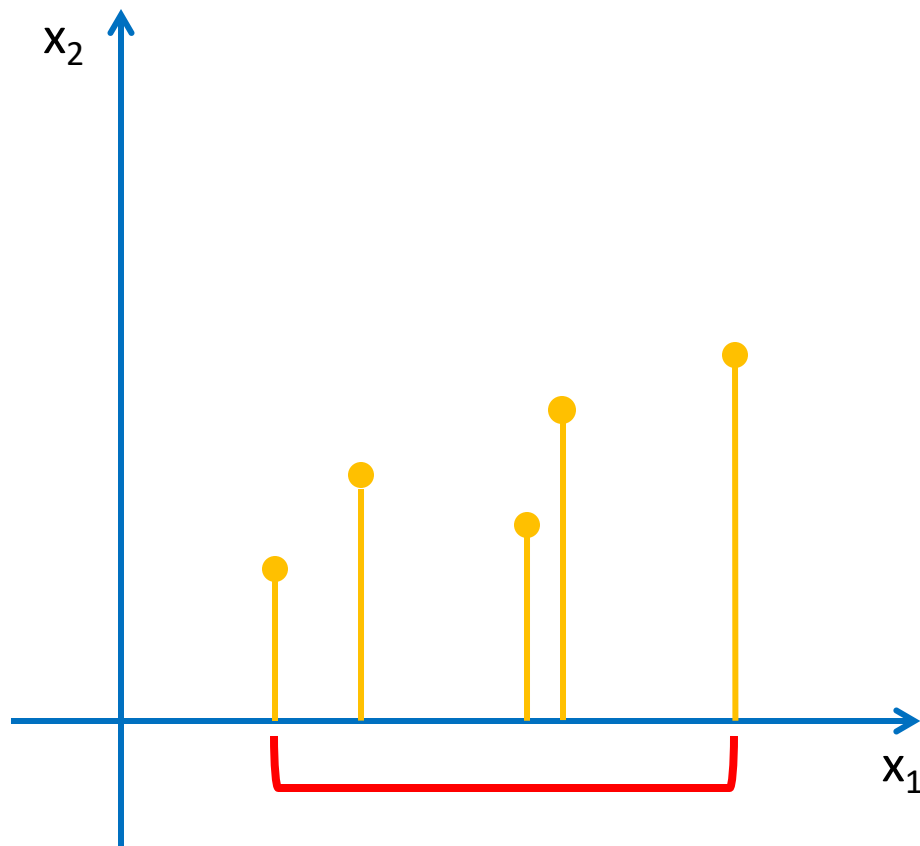
$$(x_1^1, x_2^1)$$

$$(x_1^2, x_2^2)$$

$$(x_1^3, x_2^3)$$

$$(x_1^4, x_2^4)$$

$$(x_1^5, x_2^5)$$



$$(x_1^1, x_2^1)$$

$$(x_1^2, x_2^2)$$

$$(x_1^3, x_2^3)$$

$$(x_1^4, x_2^4)$$

$$(x_1^5, x_2^5)$$

什么是特征降维? --以PCA为例

- 1. 对原特征 x_1, x_2 进行变换得到主元 z_1, z_2

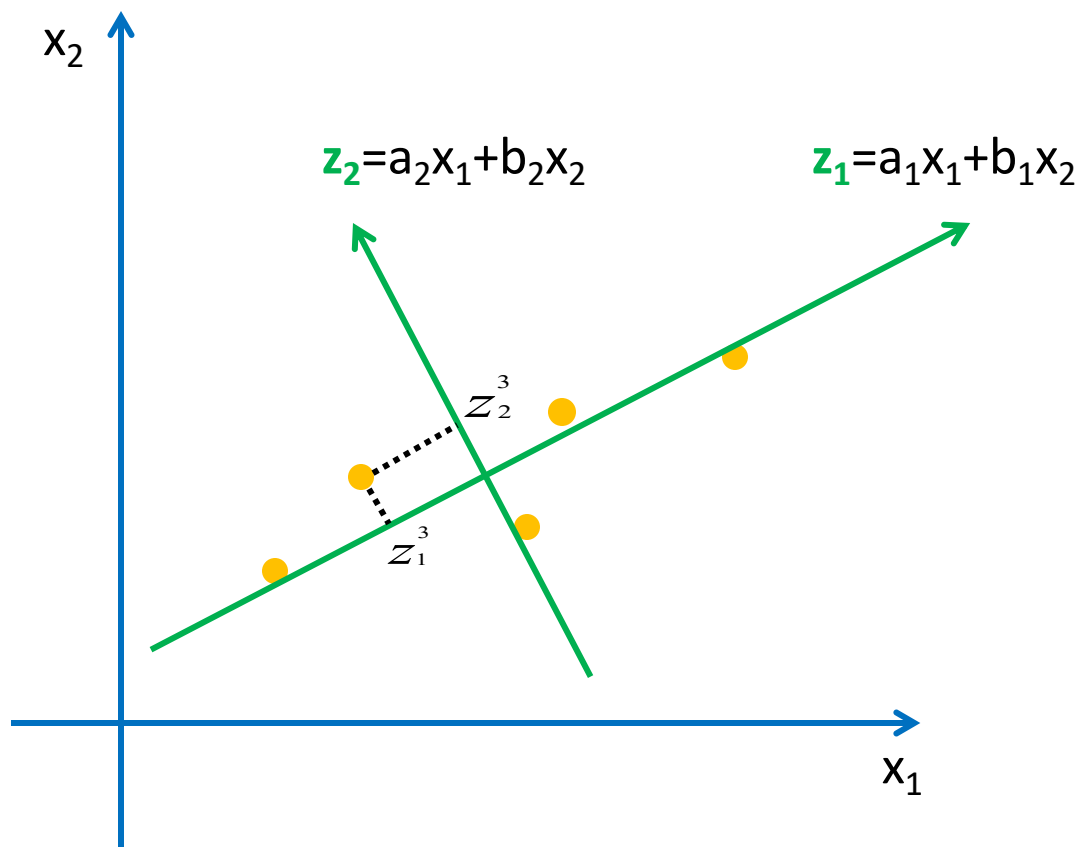
$$(x_1^1, x_2^1)$$

$$(x_1^2, x_2^2)$$

$$(x_1^3, x_2^3)$$

$$(x_1^4, x_2^4)$$

$$(x_1^5, x_2^5)$$



$$(z_1^1, z_2^1)$$

$$(z_1^2, z_2^2)$$

$$(z_1^3, z_2^3)$$

$$(z_1^4, z_2^4)$$

$$(z_1^5, z_2^5)$$

什么是特征降维? --以PCA为例

■ 2.1 保留 z_2 , 丢掉 z_1

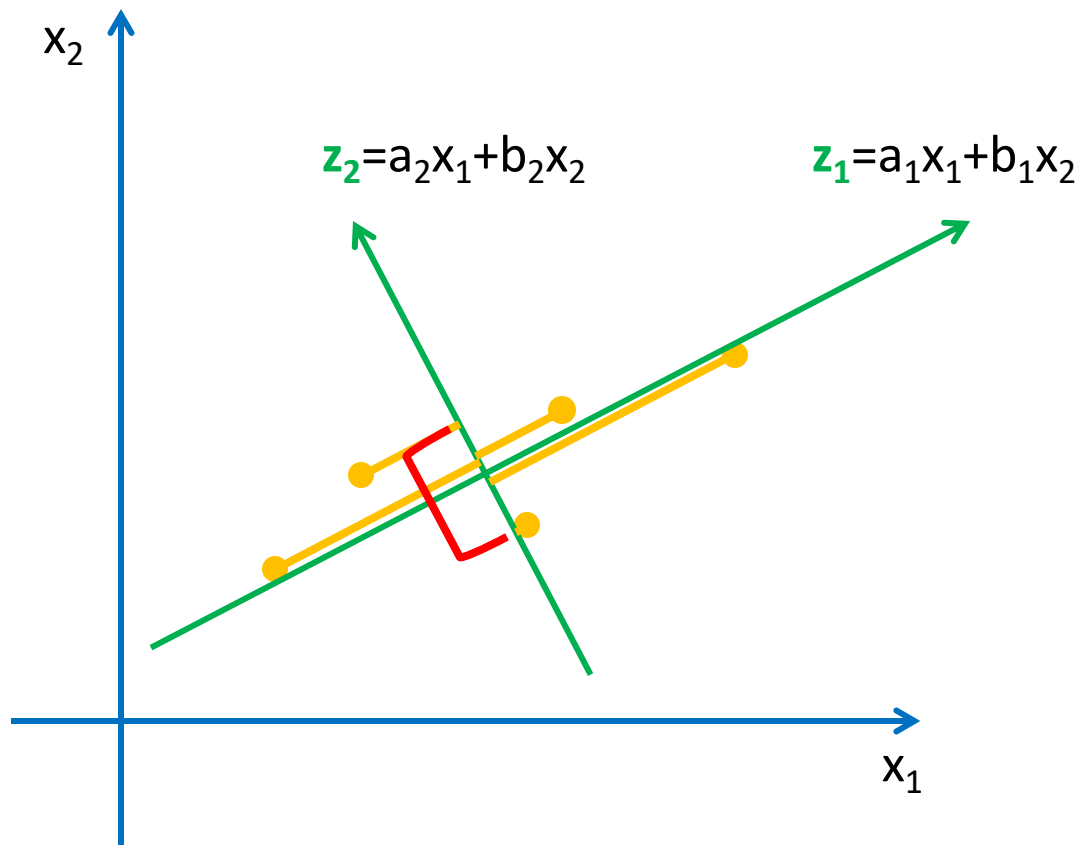
$$(x_1^1, x_2^1)$$

$$(x_1^2, x_2^2)$$

$$(x_1^3, x_2^3)$$

$$(x_1^4, x_2^4)$$

$$(x_1^5, x_2^5)$$



$$(z_1^1, z_2^1)$$

$$(z_1^2, z_2^2)$$

$$(z_1^3, z_2^3)$$

$$(z_1^4, z_2^4)$$

$$(z_1^5, z_2^5)$$

什么是特征降维？--以PCA为例

■ 2.2 保留 z_1 , 丢掉 z_2 **保留的特征不再是原来的特征**

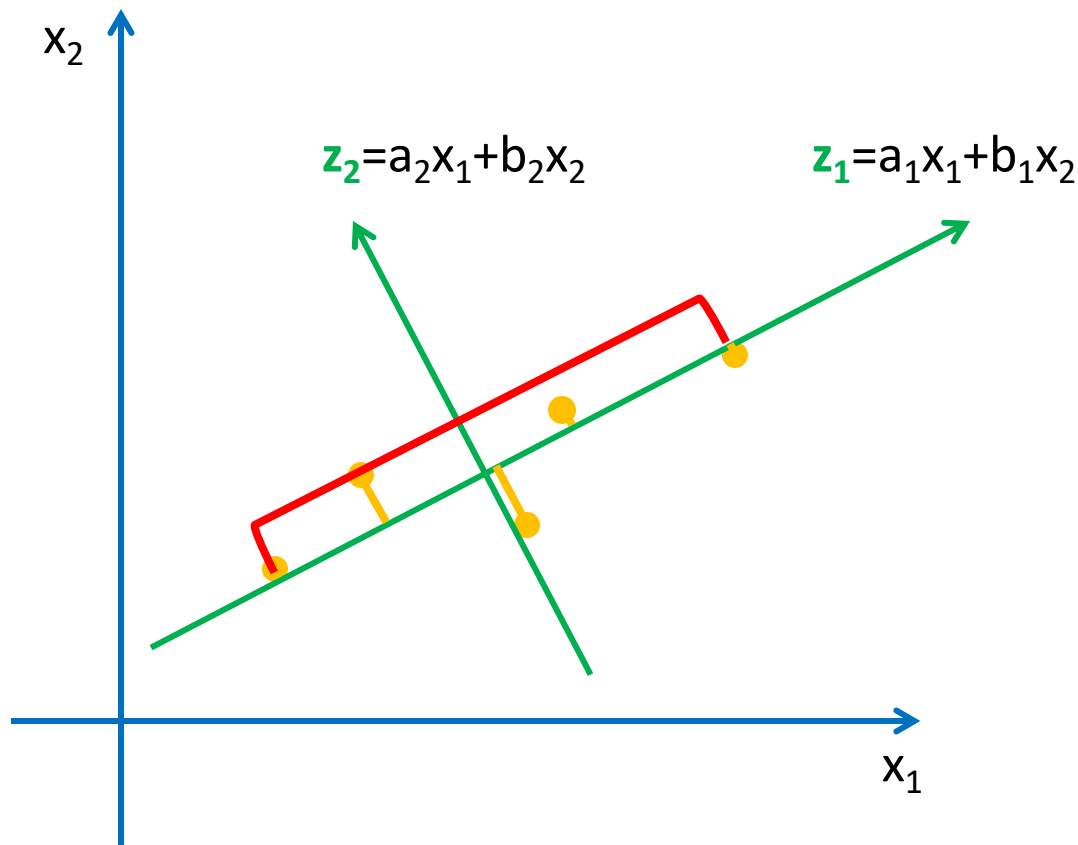
$$\begin{pmatrix} z_1^1, z_2^1 \end{pmatrix}$$

$$\begin{pmatrix} z_1^2, z_2^2 \end{pmatrix}$$

$$\begin{pmatrix} z_1^3, z_2^3 \end{pmatrix}$$

$$\begin{pmatrix} z_1^4, z_2^4 \end{pmatrix}$$

$$\begin{pmatrix} z_1^5, z_2^5 \end{pmatrix}$$



$$\begin{pmatrix} z_1^1, z_2^1 \end{pmatrix}$$

$$\begin{pmatrix} z_1^2, z_2^2 \end{pmatrix}$$

$$\begin{pmatrix} z_1^3, z_2^3 \end{pmatrix}$$

$$\begin{pmatrix} z_1^4, z_2^4 \end{pmatrix}$$

$$\begin{pmatrix} z_1^5, z_2^5 \end{pmatrix}$$

小结

本节我们主要学习了**特征选择**和**特征降维**的区别：

- **特征选择**：通过某些**指标**对原特征进行**筛选**，**保留的特征是原特征集的子集**
- **特征降维**：对原特征空间进行**变换**得到**新的**特征空间，在新的特征空间进行特征选择，**保留的特征不再是**原有特征





2

chapter

特征选择



过滤法(filter) - 单变量

- 模型建模**预测标签y**需要来自**特征x的信息**，特征x的信息量**越大越好**，什么是信息量？
- 连续变量(**方差过滤**)：**方差**越大，**信息量**越大
- 分类变量(**熵过滤**)：**熵**越大，**信息量**越大



熵或者**方差**越大，对预测 y 的**作用**就越大吗？

熵为0，对Y**没有任何意义**

方差为0，对Y**没有任何意义**

年龄 (X1)	专业 (X2)	性别 (Y)
19	数学	M
19	数学	M
19	数学	F
19	数学	F
19	数学	M
19	数学	M
19	数学	F
19	数学	F

过滤法(filter) - 双变量 (关注x和y之间的关系)

- 希望选出与**标签 y 有关**且有意义的特征，主要指标和方法：
- 皮尔逊相关性 (连续-连续)
- 均值比较 (离散-连续)
- 卡方检验 (离散-离散)
- 互信息

马爸爸(特征x)钱(方差)很多，坑爹的是跟我(标签y)没关系！



悲伤逆流成河

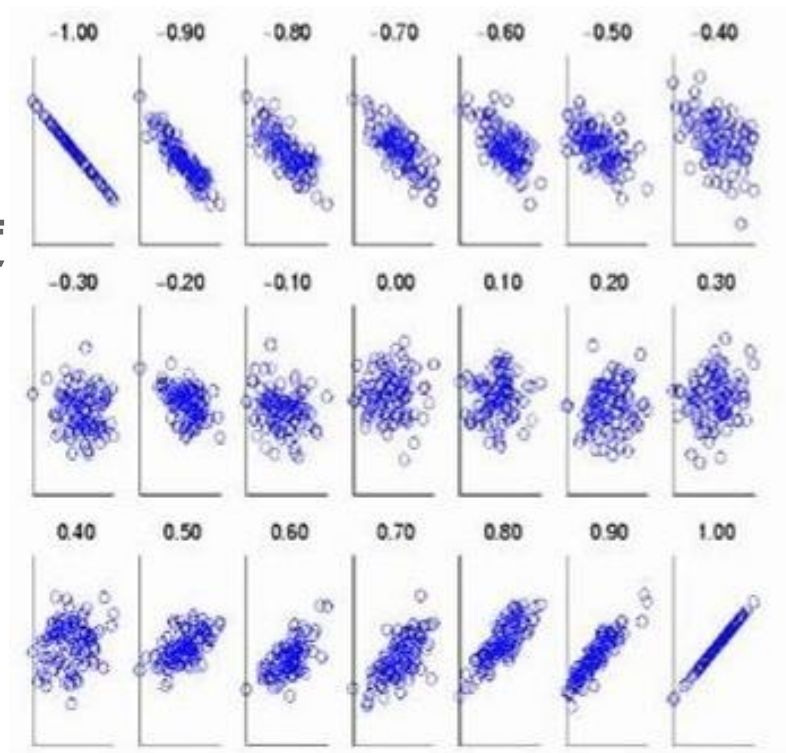
过滤法(filter) - 皮尔逊相关性

- 协方差是一个反映**两个随机变量**相关程度的指标，如果 Y 随着 X **同时变大或变小**，那么协方差就是**正值**，反之相反。

$$\text{cov}(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{n - 1}$$

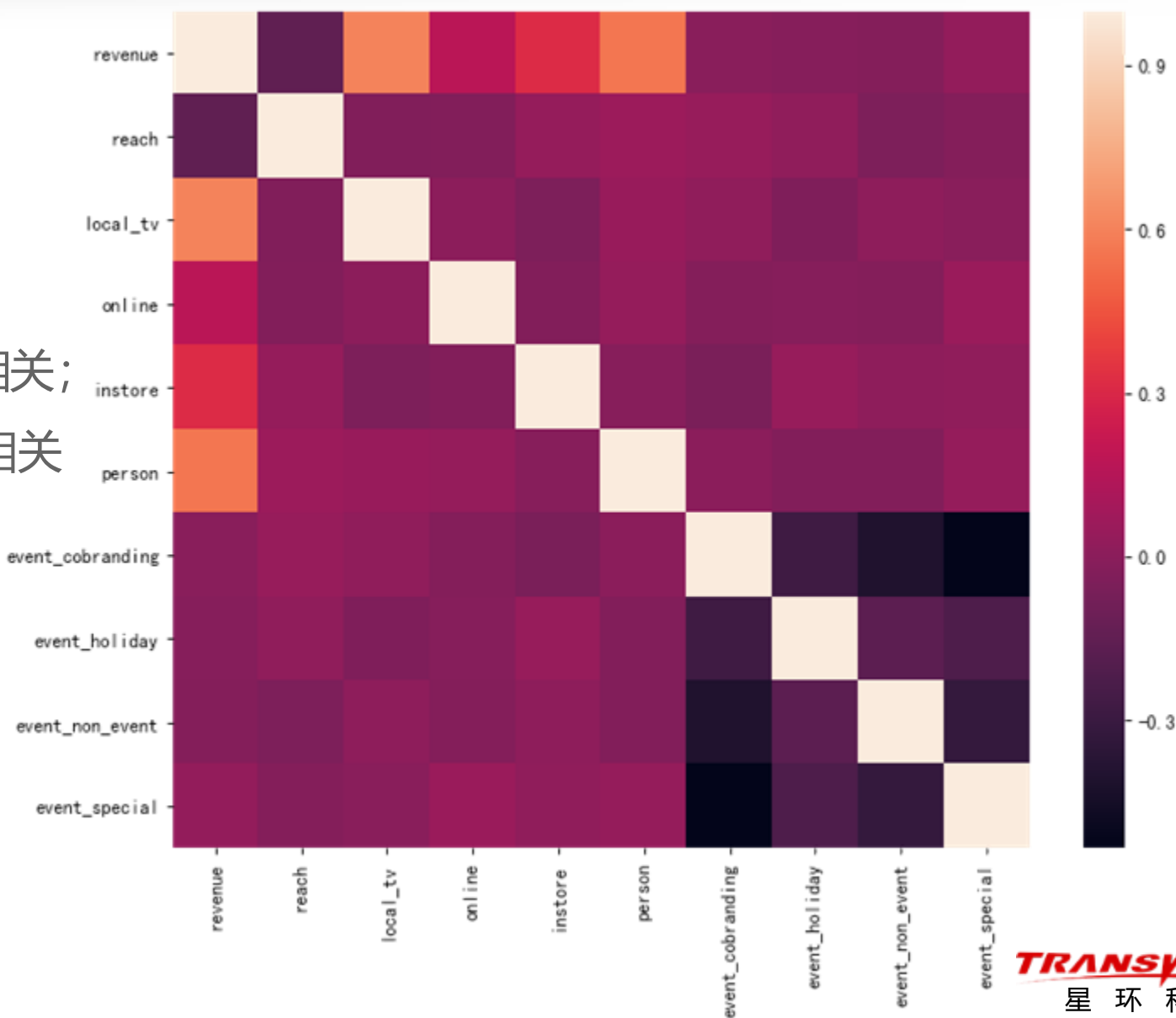
- 协方差跟X和Y本身的方差有关，其**值的大小不能**代表**相关程度**
- **皮尔逊相关系数**是协方差除以两个变量的**标准差**得到的，剔除了 X 和 Y 本身方差的影响，值域为**[-1, 1]**

$$\rho_{X,Y} = \text{corr}(X, Y) = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}$$



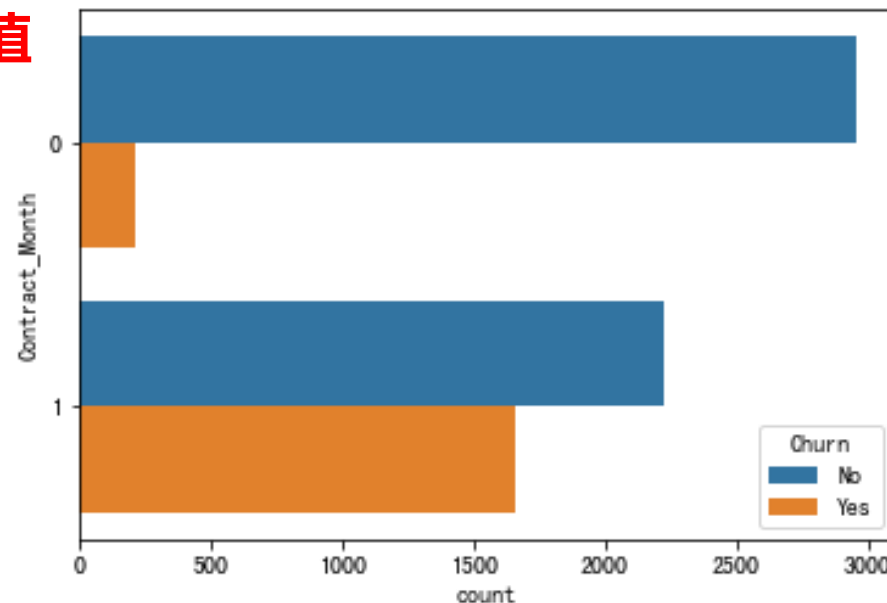
过滤法(filter) - 皮尔逊相关性

- 仅对**线性数据**有效
- 一般可按三级划分：
 - $|r| < 0.4$ 为低度相关;
 - $0.4 \leq |r| < 0.7$ 为显著性相关;
 - $0.7 \leq |r| < 1$ 为高度线性相关



过滤法(filter) - 均值比较

- 将数据按照离散变量X分组，统计每一组 Y 的平均值
 - 平均值差异较大，说明二者相关
 - 平均值差异较小，说明二者不相关
 - 使用统计学中的 Z 检验更靠谱



y	Partner_att	Dependents_att	landline	internet_att	internet_other	StreamingTV	StreamingMovies	Contract_Month	Contract_1YR	PaymentBank	Pa
Churn											
No	0.528218	0.344801	0.901044	0.379204	0.347700	0.365868	0.369927	0.429068	0.252609	0.248550	
Yes	0.357945	0.174425	0.909042	0.245586	0.693954	0.435527	0.437667	0.885500	0.088818	0.138042	

过滤法(filter) - 卡方检验、互信息

- 经典的卡方检验是检验定性（离散）自变量对定性（离散）因变量的相关性

gender	Female	Male
Churn		
No	2549	2625
Yes	939	930

$$\chi^2 = \sum \frac{(A - E)^2}{E}$$

- 互信息及最大信息系数(MIC) -- 对非线性数据有效

$$I(x; y) = \int p(x, y) \log_2 \frac{p(x, y)}{p(x)p(y)} dx dy \quad mic(x; y) = \max_{a*b < B} \frac{I(x; y)}{\log_2 \min(a, b)}$$

过滤法(filter)-加载工具包与分割数据

```
#加载数据集
from sklearn.datasets import load_iris
#用于数据集划分
from sklearn.model_selection import train_test_split
#用于标准化处理
from sklearn.preprocessing import StandardScaler
#K-近邻模型
from sklearn.tree import DecisionTreeClassifier

iris=load_iris()
X=iris.data
y=iris.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=12)
X_train[:2]
```

```
array([[5.4, 3.7, 1.5, 0.2],
       [5.1, 3.8, 1.6, 0.2]])
```

```
dt=DecisionTreeClassifier()
dt.fit(X_train,y_train)
dt.score(X_test,y_test)
```

```
0.9555555555555556
```

过滤法(filter)-方差选择法

```
from sklearn.feature_selection import VarianceThreshold
#方差选择法, 返回值为特征选择后的数据
#参数threshold为方差的阈值
vt=VarianceThreshold(threshold=0.5)
X_train_1=vt.fit_transform(X_train)
X_train_1[:2]
```

```
array([[5.4, 1.5, 0.2],
       [5.1, 1.6, 0.2]])
```

```
#对测试集进行处理并使用特征筛选后的特征进行模型训练和评估
X_test_1=vt.transform(X_test)
dt=DecisionTreeClassifier()
dt.fit(X_train_1,y_train)
dt.score(X_test_1,y_test)
```

```
0.9777777777777777
```

过滤法(filter)-相关系数法

```
from sklearn.feature_selection import SelectKBest
from scipy.stats import pearsonr
import numpy as np
#选择K个最好的特征，返回选择特征后的数据
#第一个参数为计算评估特征是否好的函数，该函数输入特征矩阵和目标向量，输出二元组（评分，P值）的数组，数组第i项为第i个特征的评分和P值。在此定义为
#参数k为选择的特征个数
sk=SelectKBest(lambda X,Y:np.array(list(map(lambda x:pearsonr(x, Y), X.T))).T[0],k=3)
X_train_2=sk.fit_transform(X_train,y_train)
X_train_2[:2]
```

```
array([[5.4, 1.5, 0.2],
       [5.1, 1.6, 0.2]])
```

```
X_test_2=sk.transform(X_test)
dt=DecisionTreeClassifier()
dt.fit(X_train_2,y_train)
dt.score(X_test_2,y_test)
```

```
0.9777777777777777
```


过滤法(filter)-互信息法

```
from minepy import MINE #1.2.5
```

```
#由于MINE的设计不是函数式的，定义mic方法将其为函数式的
```

```
def mic(x, y):  
    m = MINE()  
    m.compute_score(x, y)  
    return m.mic()
```

```
#选择K个最好的特征，返回特征选择后的数据
```

```
sk=SelectKBest(lambda X,Y:np.array(list(map(lambda x:mic(x, Y), X.T))).T,k=3)  
X_train_3=sk.fit_transform(X_train,y_train)  
X_train_3[:2]
```

```
array([[5.4, 1.5, 0.2],  
       [5.1, 1.6, 0.2]])
```

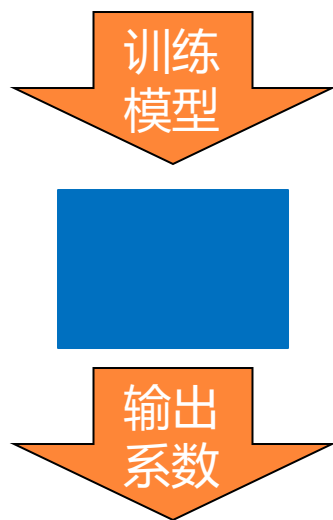
```
X_test_3=sk.transform(X_test)  
dt=DecisionTreeClassifier()  
dt.fit(X_train_3,y_train)  
dt.score(X_test_3,y_test)
```

```
0.9777777777777777
```

包装法(wrapper) - 递归特征消除法 (RFE)

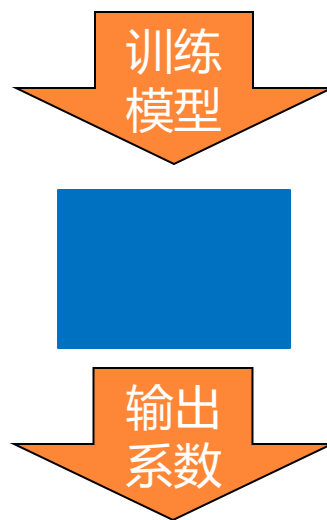
- 用一个**基模型**来进行**多轮**训练，**每**轮训练后，**消除若干权值系数的特征**，再基于**新的特征集**进行**下一轮**训练

特征集合: {A、B、C、D、E}



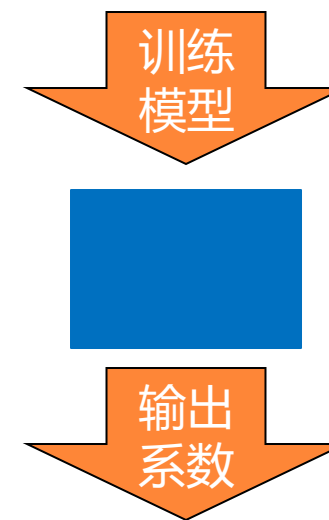
A:2.3、B:1.5、C:3.5、
D:0.2、E:1.7

特征子集: {A、B、C、E}



A:3.6、**B:2.5**、C:4.5、E:2.7

特征子集: {A、C、E}



A:3.2、C:3.5、E:1.7

包装法(wrapper)-递归特征消除法 (RFE)

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
#递归特征消除法, 返回特征选择后的数据
#参数estimator为基模型
#参数n_features_to_select为选择的特征个数
rfe=RFE(estimator=LogisticRegression(), n_features_to_select=3)
X_train_4=rfe.fit_transform(X_train,y_train)
X_train_4[:2]
```

```
array([[3.7, 1.5, 0.2],
       [3.8, 1.6, 0.2]])
```

```
X_test_4=rfe.transform(X_test)
dt=DecisionTreeClassifier()
dt.fit(X_train_4,y_train)
dt.score(X_test_4,y_test)
```

```
0.9777777777777777
```

包装法(wrapper) - 全局最优子集搜索

- 遍历每一种特征组合，选出**效果最好**的一种组合

特征集合：{A、B、C、D、E}

特征
子集

{A}
{A,B}
{A,C}
...
{A,B,C,D,E}
共 **2^5-1** 个

特征**较多**的时候**计算量大**到无法承受
推荐使用**局部最优子集搜索**

训练 **2^5-1** 个模型，搜索出能够让模型效果最好的一种组合

{A,C,E}

包装法(wrapper)-全局最优子集搜索

```
from itertools import combinations
#定义最优子集搜索类
class BestSubset():
    def __init__(self, model, X, y, random_state=None):
        self.model=model
        self.X=X
        self.y=y
        self.num_features=X.shape[1]
        self.random_state=random_state
    def getSubsets(self):#生成所有的子集
        indices=list(range(self.num_features))
        indices_list = sum([list(map(list, combinations(indices, i))) for i in range(len(indices) + 1)], [])
        return indices_list[1:]
```

后半代码在下一页

包装法(wrapper)-全局最优子集搜索

```
def fit(self):  
    #分割数据  
    X_train,X_test,y_train,y_test=train_test_split(self.X,self.y,test_size=0.3,random_state=self.random_state)  
    #搜索准确率最大的特征子集  
    max_acc=0.0  
    best_indices=[]  
    indices_list=self.getSubsets()  
    for indices in indices_list:  
        X=X_train[:,indices]  
        t_X=X_test[:,indices]  
        self.model.fit(X,y_train)  
        acc=self.model.score(t_X,y_test)  
        if acc>max_acc:#判断acc是否大于之前的最大值  
            max_acc=acc  
            best_indices=indices  
    self.best_indices=best_indices  
def transform(self,X):  
    return X[:,self.best_indices]  
def fit_transform(self,X):  
    self.fit()  
    return self.transform(X)
```

包装法(wrapper)-全局最优子集搜索

```
from sklearn.ensemble import RandomForestClassifier  
bs=BestSubset(RandomForestClassifier(),X,y,random_state=1)  
bs.getSubsets() #输出所有特征子集索引
```

```
[[0],  
 [1],  
 [2],  
 [3],  
 [0, 1],  
 [0, 2],  
 [0, 3],  
 [1, 2],  
 [1, 3],  
 [2, 3],  
 [0, 1, 2],  
 [0, 1, 3],  
 [0, 2, 3],  
 [1, 2, 3],  
 [0, 1, 2, 3]]
```

```
#训练，输出最优子集索引  
bs.fit()  
bs.best_indices
```

```
[0, 2]
```


包装法(wrapper)-全局最优子集搜索

```
X_train_5=bs.transform(X_train)  
X_train_5[:2]
```

```
array([[5.4, 1.5],  
       [5.1, 1.6]])
```

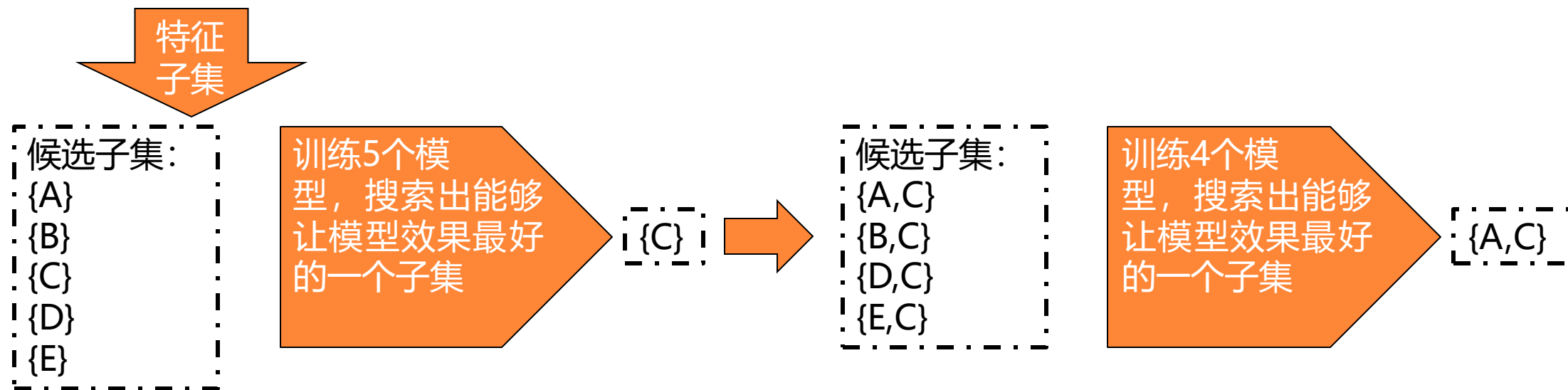
```
X_test_5=bs.transform(X_test)  
dt=DecisionTreeClassifier()  
dt.fit(X_train_5,y_train)  
dt.score(X_test_5,y_test)
```

```
0.9555555555555556
```

包装法(wrapper) - 局部最优子集搜索

- **前向**搜索：将每个特征看作一个候选子集，逐渐增加相关特征的策略。
- **后向**搜索：从完整的特征集合开始，逐渐减少无关特征的策略。
- **双向**搜索：结合前向和后向，每轮增加选定相关特征，同时减少无关特征。

特征集合：{A、B、C、D、E}



包装法(wrapper)-局部最优子集搜索

```
#定义局部最优子集搜索类
class LocalSubset():
    def __init__(self,model,X,y,random_state=None):
        self.model=model
        self.X=X
        self.y=y
        self.num_features=X.shape[1]
        self.random_state=random_state
```

后续代码在下一页

包装法(wrapper)-局部最优子集搜索

```
def fit(self):  
    #分割数据  
    X_train,X_test,y_train,y_test=train_test_split(self.X,self.y,test_size=0.3,random_state=self.random_state)  
    all_index=set(range(self.num_features))  
    #搜索准确率最大的特征子集  
    max_acc_0=0.0  
    best_indices=set()  
    for i in range(len(all_index)):  
        max_acc_1=0.0  
        best_index=None  
        for index in all_index-best_indices:  
            X=X_train[:,list(best_indices)+[index]]  
            t_X=X_test[:,list(best_indices)+[index]]  
            self.model.fit(X,y_train)  
            acc=self.model.score(t_X,y_test)  
            if acc>max_acc_1:#判断acc是否大于之前的最大值  
                max_acc_1=acc  
                best_index=index  
        #判断本轮最优特征加入后acc是否提高  
        if max_acc_1>max_acc_0:  
            max_acc_0=max_acc_1  
            best_indices.add(best_index) #将本轮的最优特征加入最有特征索引列表  
        else:#如果没有提高, 算法结束  
            break  
    self.best_indices=list(best_indices)
```

后续代码在下一页

包装法(wrapper)-局部最优子集搜索

```
def transform(self, X):  
    return X[:, self.best_indices]  
def fit_transform(self, X):  
    self.fit()  
    return self.transform(X)
```

包装法(wrapper)-局部最优子集搜索

```
ls=LocalSubset(RandomForestClassifier(),X,y,random_state=1)
```

```
#训练, 输出最优子集索引
```

```
ls.fit()
```

```
ls.best_indices
```

```
[2]
```

```
X_train_6=ls.transform(X_train)
```

```
X_train_6[:2]
```

```
array([[1.5],  
       [1.6]])
```

```
X_test_6=ls.transform(X_test)
```

```
dt=DecisionTreeClassifier()
```

```
dt.fit(X_train_6,y_train)
```

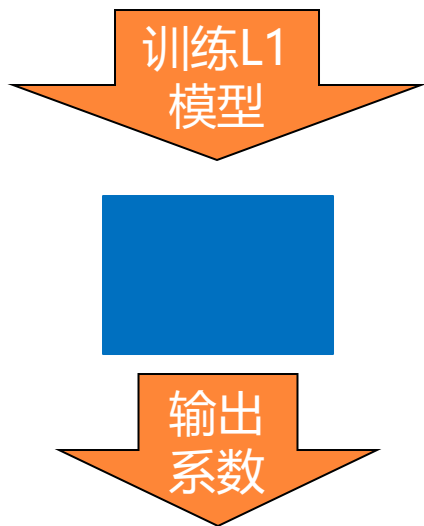
```
dt.score(X_test_6,y_test)
```

```
0.9333333333333333
```

嵌入法(embedded)

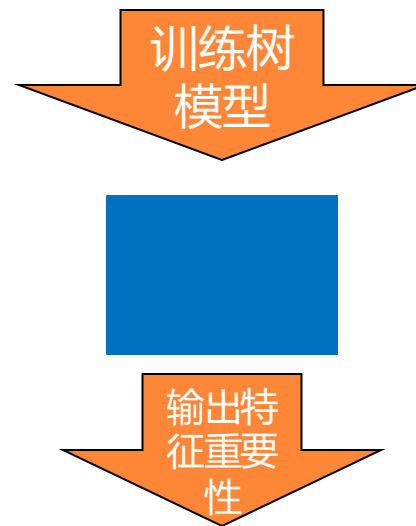
- **基于L1正则的模型**：删除模型输出**系数为0**的特征。
- **基于树的模型**：模型输出特征重要性，删除**重要性低**的特征。

特征集合：{A、 B、 C、 D、 E}



A:2.3、 **B:0**、 C:3.5、 **D:0**、 E:1.7

特征集合：{A、 B、 C、 D、 E}



A:2.3、 **B:0.3**、 C:3.5、 **D:0.6**、 E:1.7

嵌入法(embedded)-基于L1正则

```
from sklearn.feature_selection import SelectFromModel
#带L1惩罚项的逻辑回归作为基模型的特征选择
sfm=SelectFromModel(LogisticRegression(penalty="l1", C=0.1, solver='liblinear'))
X_train_7=sfm.fit_transform(X_train,y_train)
X_train_7[:2]
```

```
array([[5.4, 3.7, 1.5],
       [5.1, 3.8, 1.6]])
```

```
X_test_7=sfm.transform(X_test)
dt=DecisionTreeClassifier()
dt.fit(X_train_7,y_train)
dt.score(X_test_7,y_test)
```

```
0.9555555555555556
```


嵌入法(embedded)-基于树模型

```
from sklearn.ensemble import GradientBoostingClassifier
#GBDT作为基模型的特征选择
sfm=SelectFromModel(GradientBoostingClassifier())
X_train_8=sfm.fit_transform(X_train,y_train)
X_train_8[:2]
```

```
array([[1.5, 0.2],
       [1.6, 0.2]])
```

```
X_test_8=sfm.transform(X_test)
dt=DecisionTreeClassifier()
dt.fit(X_train_8,y_train)
dt.score(X_test_8,y_test)
```

```
0.9555555555555556
```

小结

本节我们主要学习了**特征选择**方法：

- **过滤法 (filter) :**
 - 单变量：方差过滤、熵过滤
 - 双变量：皮尔逊（线性）、卡方（离散）、MIC（非线性）
- **包装法 (wrapper) :**
 - 循环递归特征消除法 (RFE)
 - 最优子集搜索
 - 局部最优子集搜索
- **嵌入法 (embedded) :**
 - 基于L1的模型：系数为0
 - 基于树的模型：特征重要性





Q&A

TRANSWARP
星环科技